Week 10

Module 5 - B | Neural Networks

Dr. Bohan Zhuang

Learning Outcomes

1: Regularization for Deep Neural Networks

2: Unsupervised Learning of Deep Neural Networks

3: Dimensionality reduction – Auto-encoder and Principal Component Analysis (PCA)

4: Advanced unsupervised DL frameworks (extension)

**Extension indicates out of the scope of this unit

Regularization for Deep Neural Networks

The Power of (Deep) Neural Networks

- The model class corresponding to neural networks can represent almost any function (given some minor conditions) provided the network has a sufficiently large number of hidden neurons and parameters.
- Classification problem

  – Approximate the target decision boundary to any required precision
- Regression problem

  – Approximate the target function to any precision
- Cost

  – Large number of layers, and neurons in the hidden layers (heavy computational overhead; expensive run-time memory and energy cost)

  – Large number of parameters (heavy offline memory cost)

  – Tend to overfit the training data

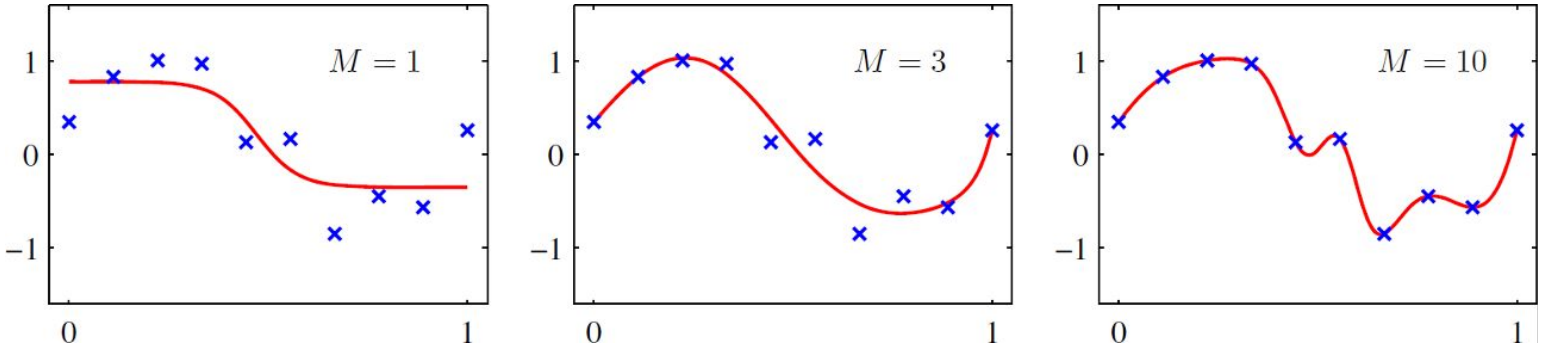
Regularization

Similar to regression, NNs can overfit the data if too many hidden neurons/parameters are added.

–Blue points: 10 data points drawn from the sinusoidal function plus some noise

–Red line: the model trained on the neural network with 1 hidden layer

> The number of hidden units is different: 1,3, and 10



Regularization

In practice, it is complicated to find the best NN for your data.

- The property of the data: the noisy level, data structure, the number of data, dimension of each sample. If data is clean, structured, low-dimensional, then the model is easy to fit and vice versa.
- The design of the model: including the number of parameters, the complexity of the operations such as the connections across neurons, the activation functions, etc.
- Optimizer: SGD, Adam, make your model converge to different local minima, leading to distinct convergence and generalisation performance.

Use cross-validation for selection

Regularization methods

- Parameter norm penalties, e.g., l2 and l1 norm
- Norm Penalties as Constrained Optimization
- Dataset Augmentation
- Noise Robustness
- Semi-Supervised Learning
- Multitask Learning
- Early Stopping
- Parameter Sharing (Clustering)
- Sparse Representations (Network Pruning)
- Low-precision Quantization

https://www.deeplearningbook.org/contents/regularization.html

Parameter Regularization

Many regularization approaches are based on limiting the capacity of models, such as neural networks, linear regression, or logistic regression, by adding a parameter norm penalty $\Omega(\boldsymbol{\theta})$ to the objective function J. We denote the regularized objective function by $\tilde{J}$:

$$\tilde{J}(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) = J(\boldsymbol{\theta}; \boldsymbol{X}, \boldsymbol{y}) + \alpha\Omega(\boldsymbol{\theta}), \tag{7.1}$$

where $\alpha \in [0, \infty)$ is a hyperparameter that weights the relative contribution of the norm penalty term, Ω, relative to the standard objective function J. Setting α to 0 results in no regularization. Larger values of α correspond to more regularization.

https://www.deeplearningbook.org/contents/regularization.html

L2 vs. L1 norm

L2 norm:

$$\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \frac{\alpha}{2}\boldsymbol{w}^\top \boldsymbol{w} + J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}), \tag{7.2}$$

with the corresponding parameter gradient

$$\nabla_{\boldsymbol{w}} \tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha \boldsymbol{w} + \nabla_{\boldsymbol{w}} J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}). \tag{7.3}$$

L1 norm:

$$\tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha ||\boldsymbol{w}||_1 + J(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}), \tag{7.19}$$

with the corresponding gradient (actually, sub gradient)

$$\nabla_{\boldsymbol{w}} \tilde{J}(\boldsymbol{w}; \boldsymbol{X}, \boldsymbol{y}) = \alpha \text{sign}(\boldsymbol{w}) + \nabla_{\boldsymbol{w}} J(\boldsymbol{X}, \boldsymbol{y}; \boldsymbol{w}), \tag{7.20}$$

where $\text{sign}(\boldsymbol{w})$ is simply the sign of $\boldsymbol{w}$ applied element-wise.

Please refer to handwritten note for comparison.

https://www.deeplearningbook.org/contents/regularization.html

L2 regularization is equivalent to imposing a Gaussian prior on the model weights

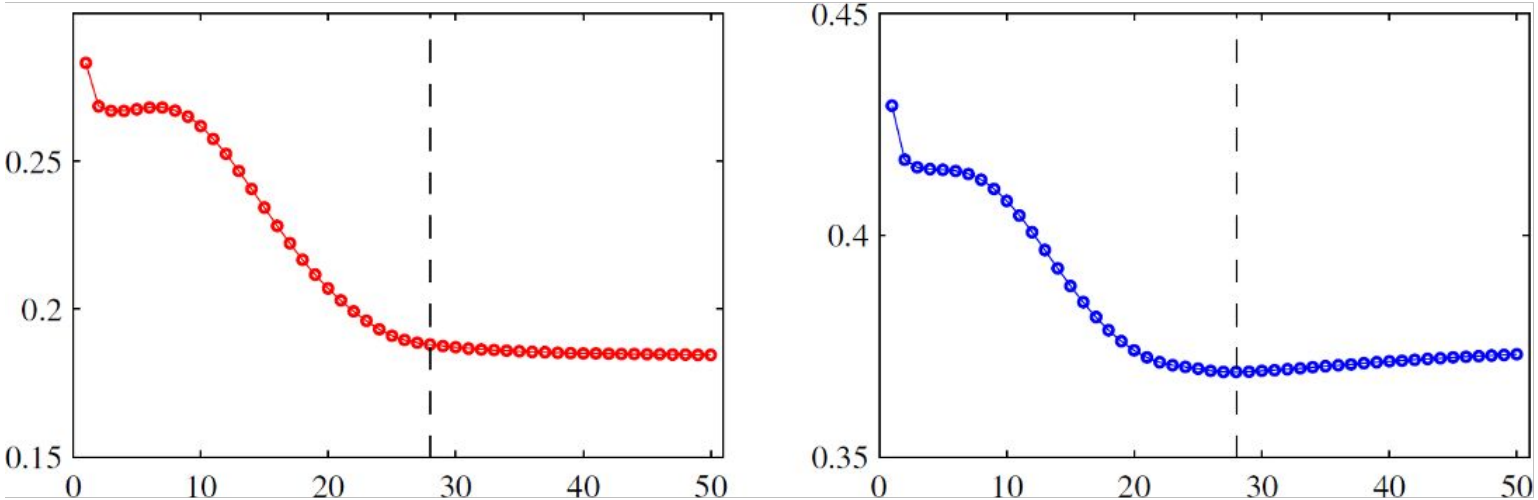
Pls refer to the handwritten note for detailed derivation.

It implies that the L2 regularization is equivalent to adding a Gaussian prior to the weights with zero mean and variance proportional to the inverse of the regularization scale.

Early Stopping

You can try TensorBoard for monitoring training dynamics

- Stop the training when the network is not learning anymore
- Stop at the point where the validation set error starts to increase
  - Indicates overfitting



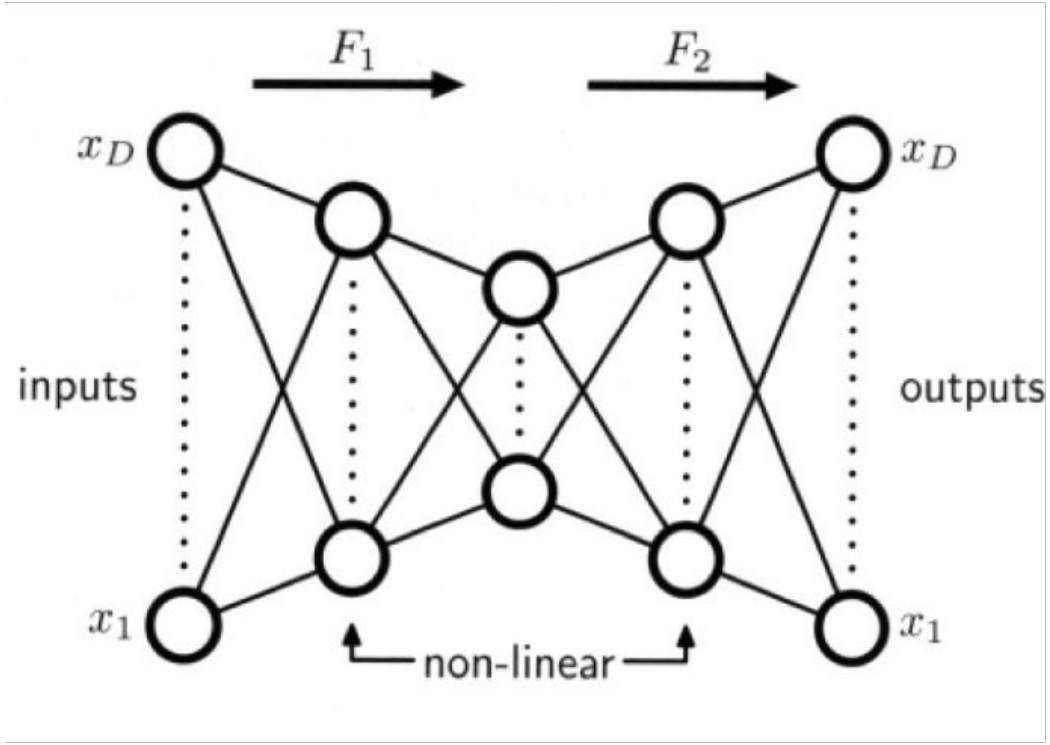
Prechelt, Lutz; Genevieve B. Orr (2012-01-01). “Early Stopping - But When?”. In Gregoire Montavon, Klaus-Robert Muller (eds.) Neural Networks: Tricks of the Trade. Lecture Notes in Computer Science. Springer Berlin Heidellberg. Pp. 53-67. ISBN 978-3-642-35289-8. Retrieved 2013-12-15

Unsupervised Learning with Neural Networks

Unsupervised Learning with Neural Networks

In unsupervised learning, you only have unlabelled data, where no ground truth labels are available during training. The NN directly learns knowledge from the data.

Autoencoder



- Number of output neurons is the same as the number of input neurons

- Optimize the weights so that the input pattern is also at the output. In other words, output is learnt to reconstruct the input.

F_1 is the encoder and F_2 is the decoder

Dimensionality reduction

Dimensionality Reduction

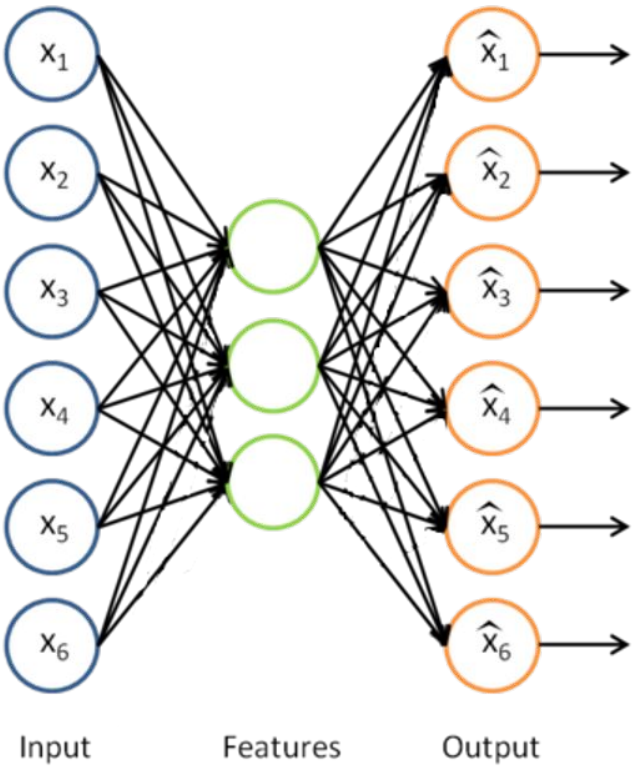
- Autoencoder
- Principal Component Analysis (PCA)
- Factor analysis (extension)
- Manifold learning (e.g., kernel PCA, Isomap, multi-dimensional scaling, etc) (extension)
- Word embeddings (extension)

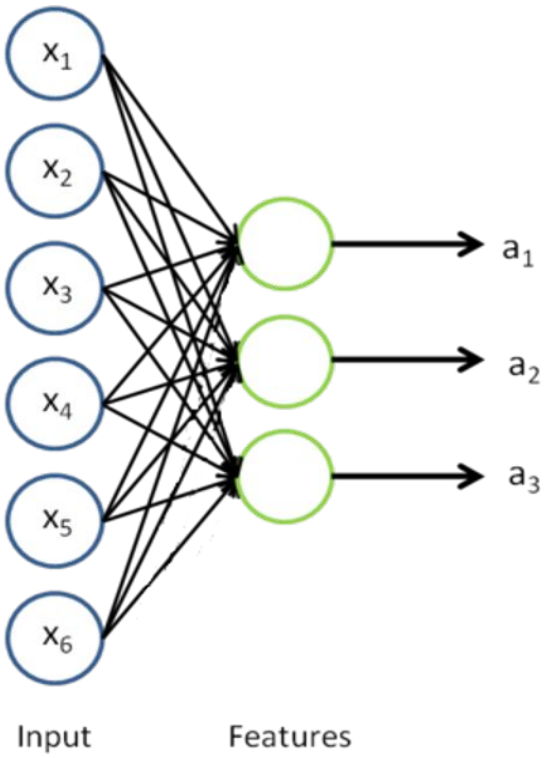
You can read Kelvin Murphy or Bishop's books.

https://probml.github.io/pml-book/book1.html

extension means out of the unit scope

Autoencoder





Trained encoder can now be used in training a classifier:

1: Use $\boldsymbol{a}$ to replace the original input $\boldsymbol{x}$

2: Concatenate $\boldsymbol{a}$ and $\boldsymbol{x}$, and use the concatenation to replace $\boldsymbol{x}$.

Learn the features from the autoencoder

Principal Component Analysis

Aim: To find the low-dimensional subspace where the data approximately lies

Method: Find the optimal basis $\{\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_k\}$ that spans the k-dimensional subspace, where k << n and n is the original dimension, $\mathbf{v}_i \in \mathbb{R}^n$.

**You need to be familiar with the basic concept of linear independence, basis, span, dimension and subspace.

Strongly recommend Gilbert Strang's Linear Algebra textbook and open course.

MIT OpenCourseWare: https://www.youtube.com/watch?v=ZK3O402wf1c&list=PL49CF3715CB9EF31D&index=1

Book: https://math.mit.edu/~gs/linearalgebra/

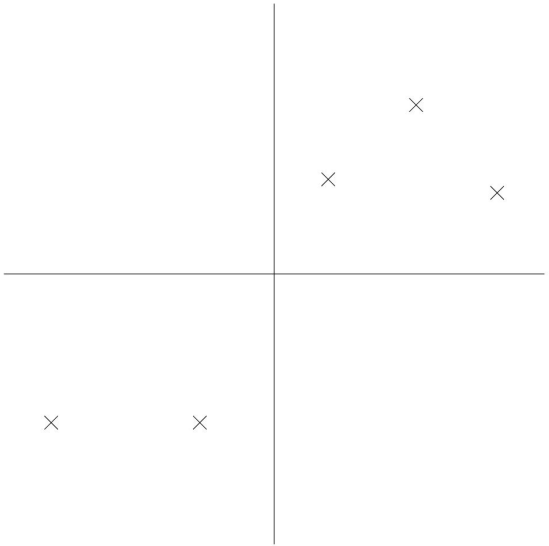
Principal Component Analysis

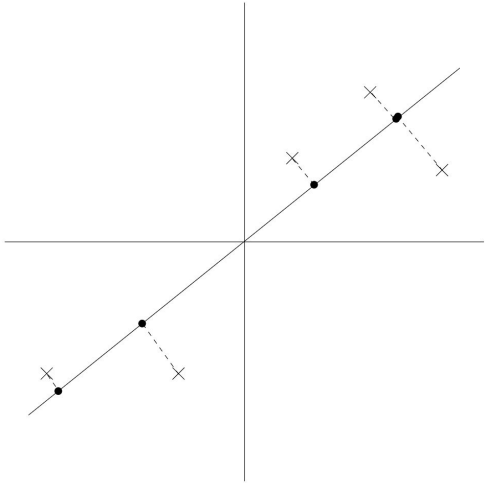
Preparation: normalize the dataset $\left\{x^{(i)}; i = 1, \ldots, m\right\}$ to zero mean and unit variance, which ensures different dimensions are in the same scale.

1. Let $\mu = \frac{1}{m}\sum_{i=1}^{m} x^{(i)}$
2. Replace each with $x^{(i)} - \mu$
3. Let $\sigma_j^2 = \frac{1}{m}\sum_{i}\left(x_j^{(i)}\right)^2$
4. Replace each $x_j^{(i)}$ with $x_j^{(i)}/\sigma_j$

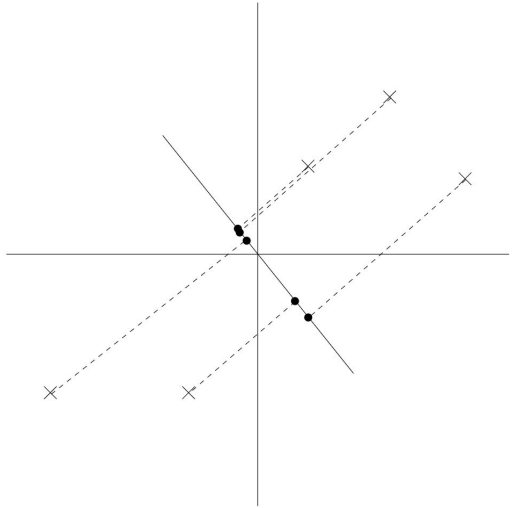
Geometry explanation

The prominent way to understand PCA is from the geometric view.





large variance



Small variance

Optimization problem: find the basis that maximizes the variance of the projections

$$\frac{1}{m}\sum_{i=1}^{m}\left(x^{(i)^T}u\right)^2 = \frac{1}{m}\sum_{i=1}^{m} u^T x^{(i)} x^{(i)^T} u$$

$$= u^T\left(\frac{1}{m}\sum_{i=1}^{m} x^{(i)} x^{(i)^T}\right) u$$

$$= u^T \Sigma u$$

see the handwritten note

, s.t. $\|u\|_2 = 1$

Absorb the constraint into the objective function by introducing the Lagrangian multiplier!!

$$L(u,\lambda) = u^T\Sigma u + \lambda(1 - u^T u)$$

Take the gradient wrt u, we get $\boxed{\Sigma u = \lambda u}$

If we wish to find a 1-dimensional subspace to approximate the data, we should choose u to be the principal eigenvector of Σ. More generally, if we wish to project our data into a k-dimensional subspace (k < n), we should choose u_1, . . . , u_k to be the top k eigenvectors of Σ. The $\{u_i\}$ now forms a new, orthogonal basis for the data.

Then, to represent $x^{(i)}$ in this basis, we need only compute the corresponding vector

$$y^{(i)} = \begin{bmatrix} u_1^T x^{(i)} \\ u_2^T x^{(i)} \\ \vdots \\ u_k^T x^{(i)} \end{bmatrix} \in \mathbb{R}^k.$$

As $x^{(i)} \in \mathbb{R}^n$, the vector $y^{(i)}$ now gives a lower, k-dimensional, approximation for $x^{(i)}$. The vectors (basis) $u_1, \ldots, u_k$ are called the first k principal components of the data.

Principal Component Analysis (PCA)

Alternatively, we can understand PCA from the singular value decomposition (SVD) perspective. Please refer to the lecture given by Prof. Gilbert Strang

https://www.youtube.com/watch?v=mBcLRGuAFUk

PCA vs. Autoencoders

- PCA is essentially a linear transformation but auto-encoders are capable of modelling complex non-linear functions.
- PCA features are totally linearly uncorrelated with each other since features are projections onto the orthogonal basis. But autoencoder's features might have correlations since they are just trained for accurate reconstruction.
- PCA is faster and computationally cheaper than autoencoders.
- A single hidden layer autoencoder with a linear activation function is very similar to PCA.
- Autoencoder is prone to overfitting due to high number of parameters. (though regularization and careful design can avoid this)

https://towardsdatascience.com/autoencoders-vs-pca-when-to-use-which-73de063f5d7

Extensions for Unsupervised DL frameworks

**The following contents are out of scope but it is good to know

Masked Learning

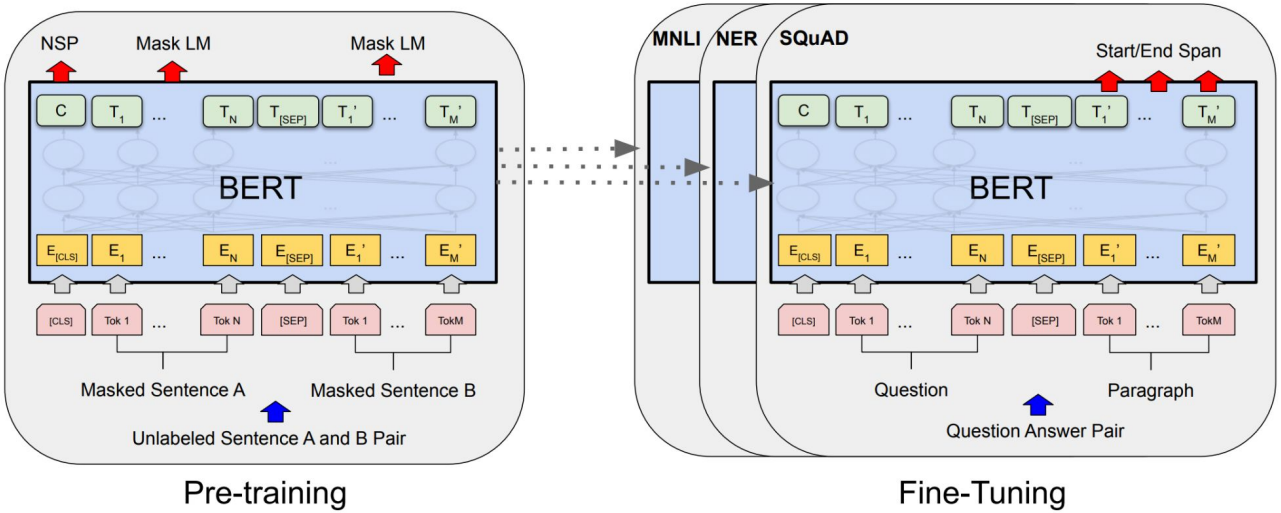


Figure 1: Overall pre-training and fine-tuning procedures for BERT. Apart from output layers, the same architectures are used in both pre-training and fine-tuning. The same pre-trained model parameters are used to initialize models for different down-stream tasks. During fine-tuning, all parameters are fine-tuned. `[CLS]` is a special symbol added in front of every input example, and `[SEP]` is a special separator token (e.g. separating questions/answers).

[BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](), in NAACL 2019.

Masked Learning

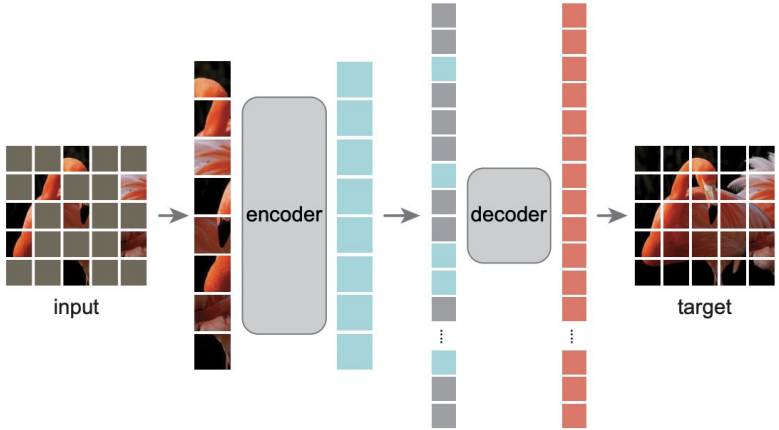


Figure 1. **Our MAE architecture**. During pre-training, a large random subset of image patches (*e.g.*, 75%) is masked out. The encoder is applied to the small subset of *visible patches*. Mask tokens are introduced *after* the encoder, and the full set of encoded patches and mask tokens is processed by a small decoder that reconstructs the original image in pixels. After pre-training, the decoder is discarded and the encoder is applied to uncorrupted images (full sets of patches) for recognition tasks.

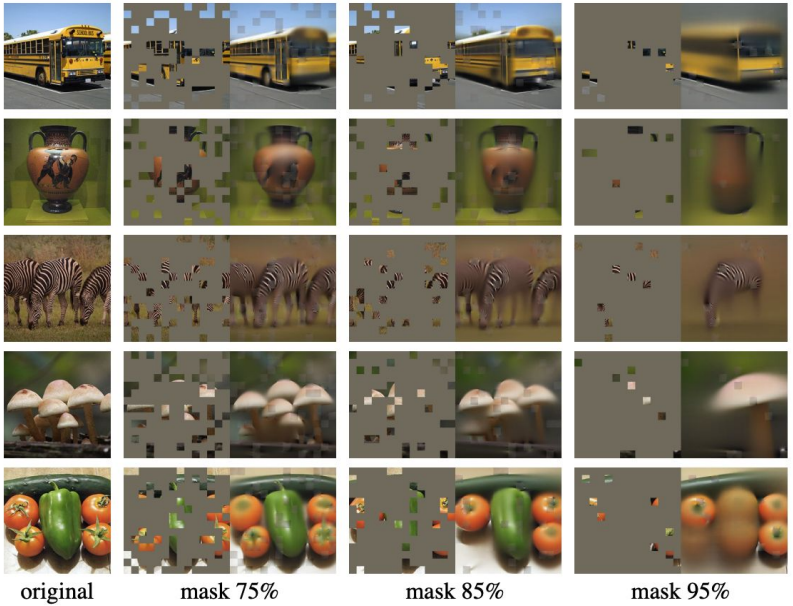
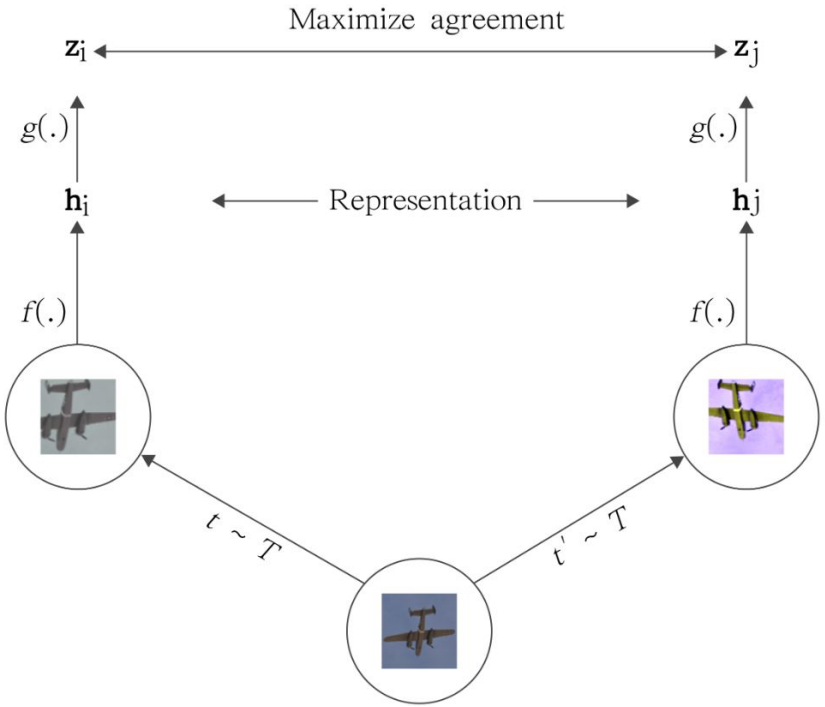


Figure 4. Reconstructions of ImageNet *validation* images using an MAE pre-trained with a masking ratio of 75% but applied on inputs with higher masking ratios. The predictions differ plausibly from the original images, showing that the method can generalize.

"Masked Autoencoders Are Scalable Vision Learners", in CVPR 2022.

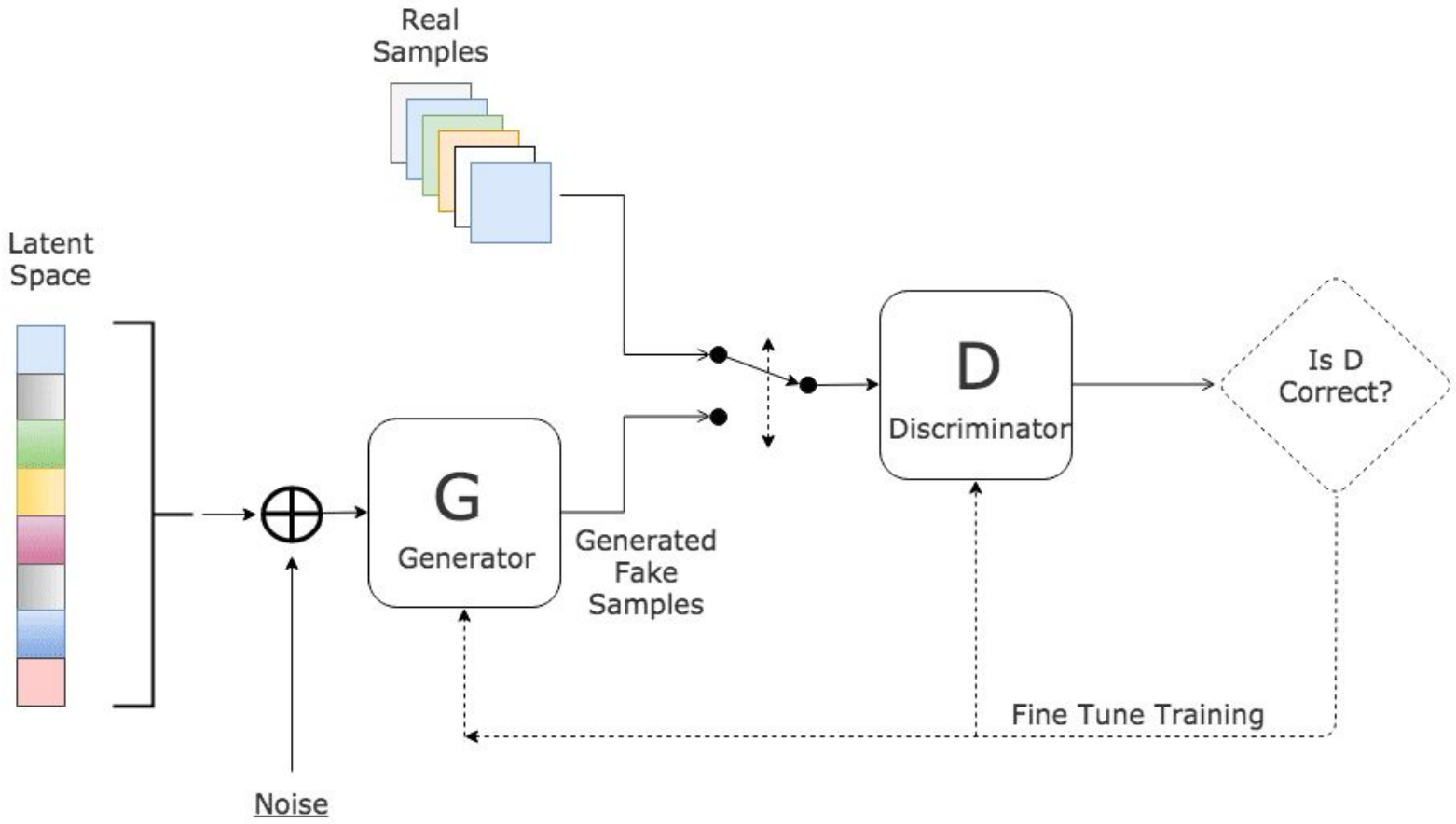
Contrastive Learning



1: https://sthalles.github.io/simple-self-supervised-learning/
2: A Simple Framework for Contrastive Learning of Visual Representations, in ICML 2020.

Generative Adversarial Network

https://www.kdnuggets.com/2017/01/generative-adversarial-networks-hot-topic-machine-learning.html



Generative Adversarial Network (GAN)

https://lilianweng.github.io/posts/2017-08-20-gan/

Symbol	Meaning	Notes
p_z	Data distribution over noise input z	Usually, just uniform.
p_g	The generator's distribution over data x	
p_r	Data distribution over real sample x	

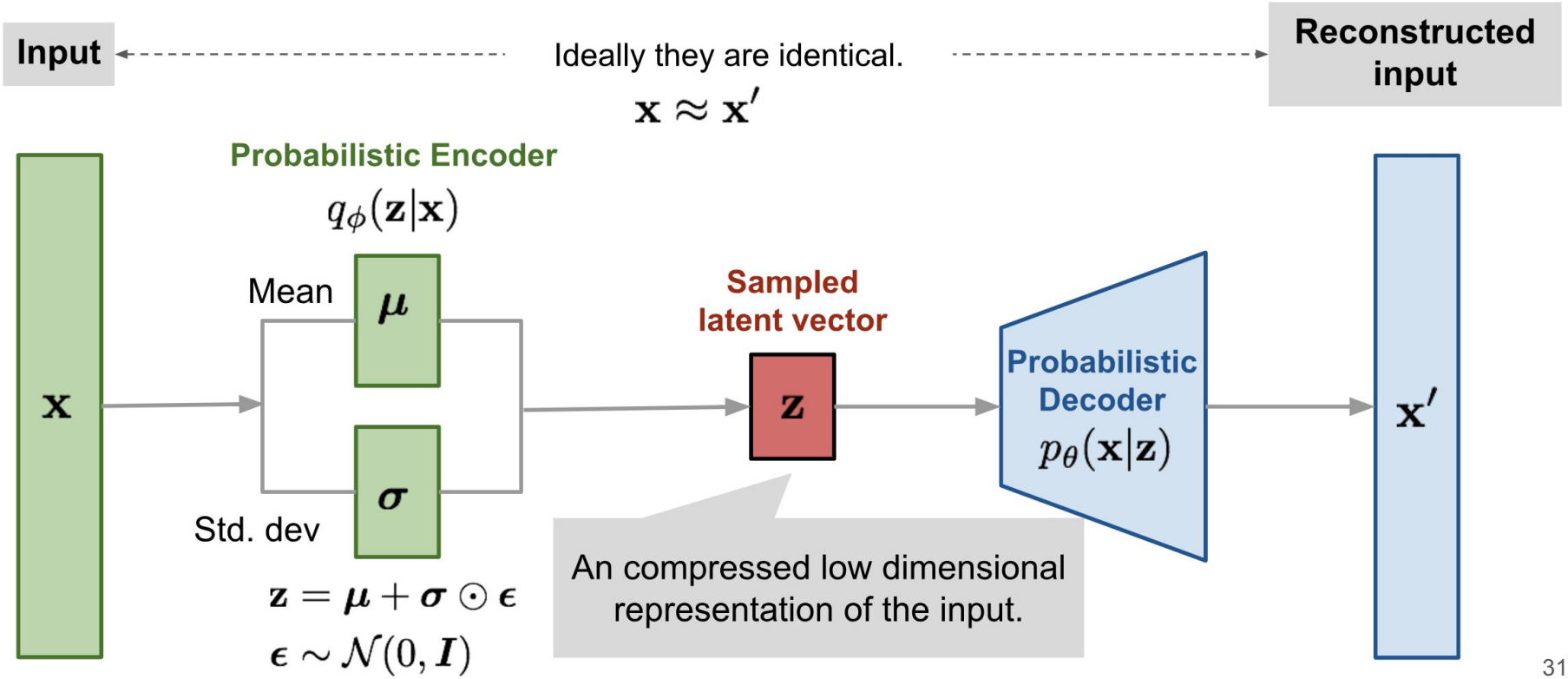
On one hand, we want to make sure the discriminator D's decisions over real data are accurate by maximizing $\mathbb{E}_{x \sim p_r(x)}[\log D(x)]$. Meanwhile, given a fake sample $G(z), z \sim p_z(z)$, the discriminator is expected to output a probability, $D(G(z))$, close to zero by maximizing $\mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$.

On the other hand, the generator is trained to increase the chances of D producing a high probability for a fake example, thus to minimize $\mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$.

When combining both aspects together, D and G are playing a **minimax game** in which we should optimize the following loss function:

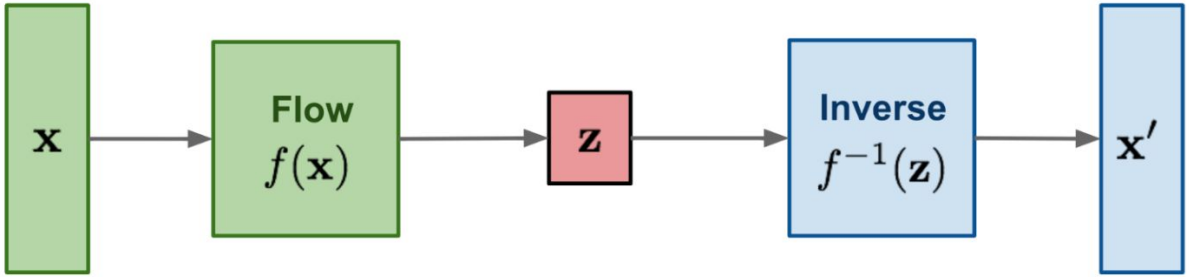
$$\begin{aligned}\min_G \max_D L(D, G) &= \mathbb{E}_{x \sim p_r(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))] \\ &= \mathbb{E}_{x \sim p_r(x)}[\log D(x)] + \mathbb{E}_{x \sim p_g(x)}[\log(1 - D(x)]\end{aligned}$$

Variational Autoencoder



Flow-based Model

Flow-based generative models: minimize the negative log-likelihood

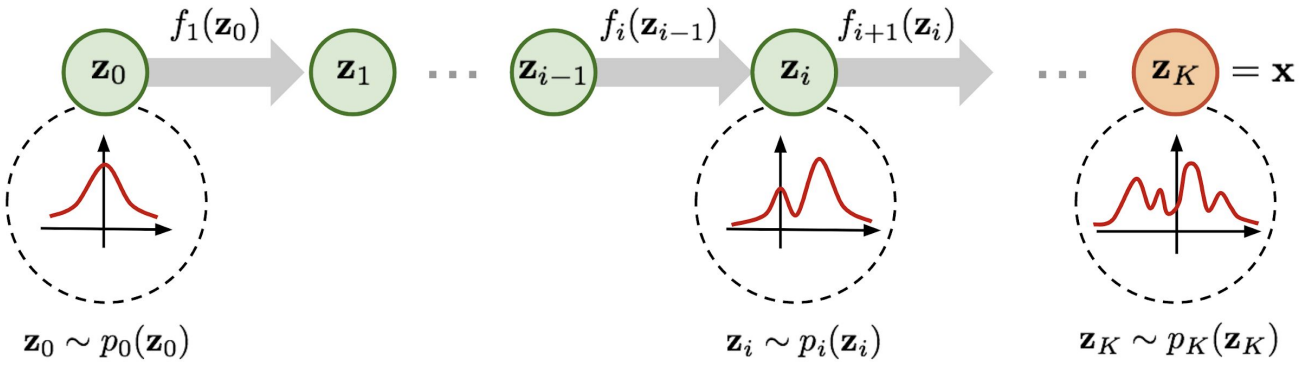


Flow-based Model

$$\int p(x)dx = \int \pi(z)dz = 1 \text{ ; Definition of probability distribution.}$$

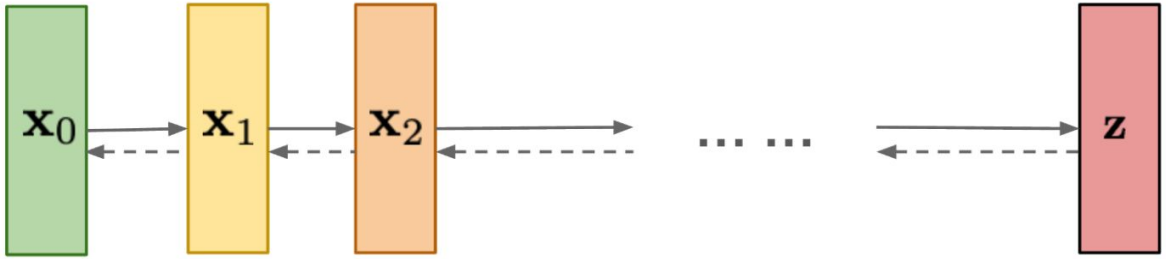
$$p(x) = \pi(z)\left|\frac{dz}{dx}\right| = \pi(f^{-1}(x))\left|\frac{df^{-1}}{dx}\right| = \pi(f^{-1}(x))|(f^{-1})'(x)|$$

Change of Variable Theorem

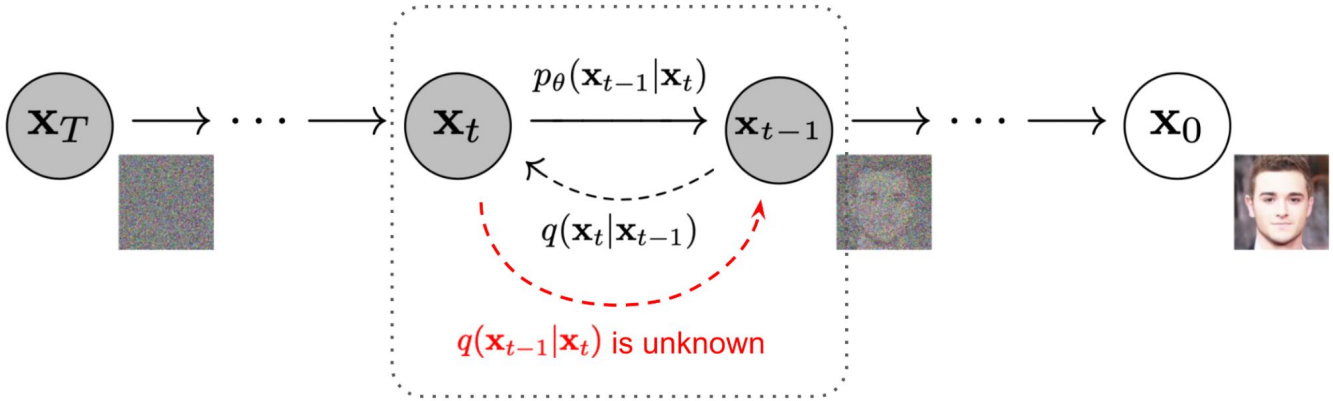


Diffusion Models

Diffusion models:
Gradually add Gaussian noise and then reverse



Use variational lower bound



The Markov chain of forward (reverse) diffusion process of generating a sample by slowly adding (removing) noise.

Diffusion Models

Forward diffusion process:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \quad q(\mathbf{x}_{1:T}|\mathbf{x}_0) = \prod_{t=1}^{T} q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

Backward diffusion process:

$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T)\prod_{t=1}^{T} p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) \quad p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t))$$

https://lilianweng.github.io/posts/2021-07-11-diffusion-models/

https://huggingface.co/spaces/stabilityai/stable-diffusion

Thank you